

From a Physics Model to a Computational Experiment

A numerical result is useful only when its model, units, assumptions, and checks are visible.

Physical question. A radioactive sample has a half-life of 6.0 h. How can we simulate its decay over one day, and how small must the time step be before we trust the result?

Learning outcomes

By the end of this week, you should be able to:

1. turn a physical question into variables, assumptions, equations, and a numerical procedure;
2. check dimensions and limiting cases before running a model;
3. implement the idea of time stepping in MATLAB and plot a result with meaningful axes and units;
4. compare a numerical result with a reference solution and explain the effect of the time step; and
5. make a computational experiment reproducible.

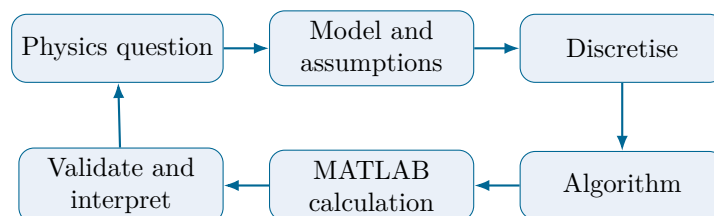
How to use this note

This note is designed to support a two-hour lecture and a short practical activity. Read the physical question first, make a prediction, and only then run the MATLAB experiment. The purpose is not to memorise a command sequence; it is to make the model, numerical method, and validation evidence visible.

Week 1 bridge. We use the radioactive-decay problem to practise the minimum MATLAB workflow: name the quantities, write a script, store values in arrays, repeat a time-step rule, plot the result, and test whether the answer deserves trust.

1. The computational-physics cycle

Computation is not a replacement for physics. It is a controlled experiment performed on a mathematical model.



At every stage, write down what you know and what you are approximating. A graph alone is not evidence that a simulation is correct.

2. MATLAB essentials for this experiment

MATLAB provides several views of the same computational experiment:

- the **Command Window** is useful for trying one expression;
- the **Editor/Live Editor** stores a repeatable script with text, code, and output;

- the **Workspace** shows variables currently in memory; and
- the **Current Folder** identifies the files MATLAB can run without an explicit path.

Use meaningful names and make units visible in the name or in a nearby comment. For this week, the essential MATLAB pattern is:

MATLAB idea	Example and purpose
Assignment	<code>radius_m = 0.02;</code> stores a value in a named variable.
Arithmetic	<code>area_m2 = pi*radius_m^2;</code> evaluates an expression.
Function	<code>lambda_per_h = log(2)/T_half_h;</code> calls a built-in function.
Comment	<code>% half-life is measured in hours</code> records a choice for a reader.
Fresh run	Start with <code>clearvars; close all;</code> so old Workspace values cannot hide a mistake.

Mini warm-up: units and expressions. The first practical contains short scalar problems. For example, convert a tank volume of 2 gallons and 4 pints to litres using 8 pints per gallon and 1.76 pints per litre:

```
1 gallons = 2;
2 pints = 4;
3 pintsPerGallon = 8;
4 pintsPerLitre = 1.76;
5 volumeLitres = (gallons*pintsPerGallon + pints)/pintsPerLitre;
6 fprintf('Volume = %.3f L\n', volumeLitres)
```

The result is approximately 11.364L. The important habits are to name the units, show the conversion, and inspect whether the magnitude is plausible.

3. Worked physical model: radioactive decay

Let $N(t)$ be the number of undecayed nuclei. For a closed sample with a constant decay constant λ ,

$$\frac{dN}{dt} = -\lambda N, \quad N(0) = N_0. \quad (1)$$

The model says that each nucleus has the same probability of decaying per unit time. It ignores, for example, a changing decay constant or nuclei entering or leaving the sample.

The half-life $T_{1/2}$ and decay constant are related by

$$\lambda = \frac{\ln 2}{T_{1/2}}. \quad (2)$$

For $T_{1/2} = 6.0 \text{ h}$, $\lambda = 0.1155 \text{ h}^{-1}$. Notice the units: λt must be dimensionless.

Parameter map. The physics symbols become MATLAB variables without using Greek letters in the variable names:

Physics	MATLAB name	Meaning and unit
N_0	<code>N0</code>	initial population, nuclei
$T_{1/2}$	<code>T_half_h</code>	half-life, hours
λ	<code>lambda_per_h</code>	decay constant, h^{-1}
$t_{\max}$	<code>tMax_h</code>	final simulated time, hours
Δt	<code>dt_h</code>	numerical step size, hours

Before computing, predict.

- If $\lambda \rightarrow 0$, what should happen to $N(t)$?
- At $t = 6.0 \text{ h}$, what fraction of N_0 should remain?
- Can a physically meaningful calculation return $N < 0$?

4. From differential equation to time-step rule

Choose equally spaced times $t_n = n\Delta t$. Approximate the derivative with a forward difference:

$$\frac{N_{n+1} - N_n}{\Delta t} \approx -\lambda N_n. \quad (3)$$

Rearranging gives the **forward Euler** update:

$$N_{n+1} = N_n (1 - \lambda \Delta t). \quad (4)$$

This equation is not the exact physics; it is an approximation to the model. It becomes more accurate as Δt decreases, but uses more computational steps.

Algorithm.

1. Choose N_0 , $T_{1/2}$, a final time $t_{\max}$, and Δt .
2. Compute $\lambda = \ln 2 / T_{1/2}$ and create a time array.
3. Set the first value, $N_1 = N_0$.
4. Apply the Euler update from one time index to the next.
5. Plot N/N_0 against t and compare it with a reference.

The following is a minimal, complete script. It deliberately keeps the model visible: the update rule appears as one line inside the loop.

Listing 1: A first Euler simulation

```
1 clearvars; close all;
2
3 N0 = 1000;
4 T_half_h = 6.0;
5 tMax_h = 24.0;
6 dt_h = 0.5;
7 lambda_per_h = log(2)/T_half_h;
8
9 t_h = 0:dt_h:tMax_h;
10 N = zeros(size(t_h));
11 N(1) = N0;
12
13 for n = 1:numel(t_h)-1
14     N(n+1) = N(n)*(1 - lambda_per_h*dt_h);
15 end
16
17 N_exact = N0*exp(-lambda_per_h*t_h);
18 plot(t_h,N/N0,'o-',t_h,N_exact,'k-','LineWidth',1.5)
19 xlabel('Time, t (h)'); ylabel('Fraction remaining, N/N_0')
20 legend('Forward Euler','Exact','Location','northeast')
21 grid on
```

Read the script in three layers: parameters and units, storage and initial condition, then the repeated update. The loop is not a new physical law; it is the computer carrying out the discretised rule.

5. A reference answer and numerical error

For this model, the analytic solution is available:

$$N_{\text{exact}}(t) = N_0 e^{-\lambda t}. \quad (5)$$

Use it as a benchmark, not as a shortcut that makes numerical work unnecessary. A useful final-time relative error is

$$\varepsilon_{\text{rel}} = \frac{|N_{\text{Euler}}(t_{\max}) - N_{\text{exact}}(t_{\max})|}{N_{\text{exact}}(t_{\max})}. \quad (6)$$

Run the same experiment with $\Delta t = 1.0$ h, 0.5 h, and 0.1 h. The result should approach the reference as the step becomes smaller. A choice with $\lambda\Delta t > 1$ is a warning sign: the Euler update can produce a negative N , which is unphysical.

For $T_{1/2} = 6.0$ h and $t_{\max} = 24$ h, the exact fraction is $e^{-\lambda t_{\max}} = 0.0625$. The following values are useful checkpoints:

Δt (h)	$\lambda\Delta t$	$N_{\text{Euler}}(24 \text{ h})/N_0$	relative error
2.0	0.2310	0.042735	31.62%
1.0	0.1155	0.052536	15.94%
0.5	0.0578	0.057505	7.99%
0.1	0.0116	0.061499	1.60%

Smaller Δt improves the approximation here, but it also increases the number of computational steps. A good numerical experiment reports both the result and the cost of obtaining it.

6. What MATLAB is doing

MATLAB stores the time points and values in arrays. The code should make the model visible rather than hiding it in unexplained commands.

Object	Meaning in this experiment
<code>t</code>	vector of times, with one consistent unit
<code>N</code>	vector of computed populations; its first element is the initial condition
<code>dt</code>	numerical resolution, not a physical parameter of the isotope
<code>lambda</code>	decay constant; use a name that does not conflict with MATLAB functions
<code>plot(t,N/N0)</code>	visual comparison of a dimensionless fraction against time

Use comments to explain choices such as the time unit and the validation test. Do not rely on the current Workspace: a good script should run correctly after a fresh MATLAB restart.

Common errors and quick checks.

Symptom	First check
The curve starts below N_0	Did you set <code>N(1) = N0</code> ?
The first update is missing	Does the loop write <code>N(n+1)</code> rather than overwrite <code>N(n)</code> ?
Values become negative	Is $\lambda\Delta t > 1$? Check the units of λ and Δt .
The plot changes between runs	Run after <code>clearvars</code> ; do not use old values.
The graph cannot be interpreted	Add a title, axis labels, units, legend, and a caption.

7. Reproducible computational experiments

A colleague should be able to reproduce your figure using the same file and inputs. Record:

- the physical parameters and their units;
- the numerical settings, especially Δt and $t_{\max}$;
- the MATLAB version or toolboxes if they matter;
- the validation method and the result of that check; and
- a clear figure title, axis labels, units, and caption.

AI with responsibility. An AI assistant may help explain an Euler update or suggest MATLAB syntax. It cannot establish that your result is physically valid. Before using AI-assisted code, check the units, reproduce a limiting case, compare with N_{exact} , and state which test you performed.

Validation checklist. Before presenting a computational result, confirm each layer:

Layer	Evidence to record
Physics	State the question, variables, assumptions, and expected trend.
Units	Check that λ and t use the same time unit, so λt is dimensionless.
Algorithm	Show the update rule and identify the initial condition and index range.
Numerics	Compare at least two timesteps and report an error or convergence measure.
Communication	Label the figure, include units, and write one sentence interpreting it.

Experiment record. Use this compact record when you run the Live Script or present your result:

Item	Record
Physical question	What quantity is being predicted?
Parameters and units	$N_0 =$ _____, $T_{1/2} =$ _____, $t_{\max} =$ _____.
Numerical settings	$\Delta t =$ _____; number of steps =_____.
Validation evidence	Reference / limiting case / timestep comparison: _____.
Interpretation	My result is / is not adequate because _____.

8. Guided practice and self-check

Work through these tasks before looking at the answer hints:

1. **Predict.** What fractions should remain after one, two, and four half-lives? Explain why the answer does not depend on N_0 .
2. **Complete.** Fill in the missing MATLAB update: $N(n+1) = N(n)*(1 - \dots)$; . State the unit of the product inside the brackets.
3. **Experiment.** Change $T_{1/2}$ to 3.0 h and compare the exact fraction at 24 hours with the original model. Keep the time unit consistent.
4. **Debug.** A classmate's script gives a negative population and an unlabeled figure. Identify two numerical issues and two communication issues.

Exit ticket. Submit three sentences: (1) the model equation and one assumption, (2) the MATLAB line that advances one time step, and (3) the validation evidence that makes your chosen Δt acceptable.

Glossary

Term	Meaning in this note
Model	Mathematical description of a physical system and its assumptions.
State variable	Quantity whose value changes during the simulation, such as $N(t)$.
Parameter	Fixed input that defines the model, such as $T_{1/2}$ or λ .
Exact solution	A reference solution obtained without the timestep approximation.
Numerical solution	An approximate solution produced by a discrete algorithm.
Timestep	The numerical interval Δt between successive computed values.
Validation	Evidence that the implementation and interpretation agree with a trusted check.
Reproducibility	The ability of another person to rerun the same experiment from the recorded inputs and file.

Three takeaways

1. **A computational result begins with a model and assumptions, not with a MATLAB command.**
2. **A time step is part of the numerical method and must be tested.**
3. **Validation and interpretation are required parts of a simulation.**

Preparation for the practical. Bring a MATLAB script or Live Script that can simulate decay for a chosen Δt . Be ready to show one plot, one limiting-case check, and one sentence explaining whether your time step is adequate.